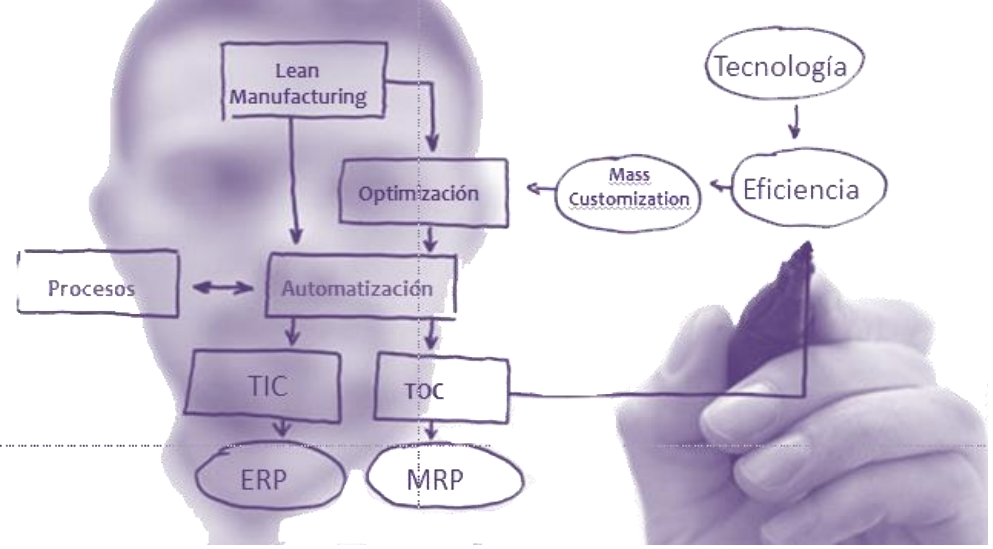


UNIVERSITAT POLITÈCNICA DE VALÈNCIA

DEIOAC-UPV

Tema 5: Resolución de Modelos de Programación Entera

Objetivos

Al finalizar el tema, deberás ser capaz de:

- Identificar la diferencia desde un punto de vista computacional entre distintas categorías de Programas Lineales.
- Conocer las principales características conceptuales de los algoritmos para la resolución de problemas de programación lineal entera.
- Interpretar los árboles que se generan al aplicar estos algoritmos, así como deducir el criterio aplicado para su obtención

© P.Tormos – A.Lova (Universitat Politècnica de València)

CONTENIDOS

5.1 Introducción

5.2 Técnicas de Programación Entera:

5.2.1 Algoritmos de Planos de Corte

5.2.2 Algoritmos de Bifurcación y Acotación

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.1 Introducción

- Muchos de los problemas reales exigen soluciones con valores enteros, por lo tanto, las variables de dicho problema deben ser definidas como variables enteras

Hipótesis	Programación Lineal	Programación Entera
H1: Divisibilidad	**SI**	**NO**
H2: No negatividad	**SI**	**SI**
H3: Linealidad	**SI**	**SI**
H4: Certidumbre	**SI**	**SI**

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.1 Introducción

- Este tipo de problemas se denominan en general Problemas de Programación Lineal Entera (PPLE). En concreto:

A. Si todas las variables del problema son enteras se habla de **PPLE Pura.**
B. Si sólo algunas son enteras y las restantes son continuas se habla de **PPLE Mixta.**
C. Si todas las variables enteras son binarias (0/1) el problema se denomina **PPLE Binaria.**

- En ingeniería los problemas más frecuentes son los PPLE Mixta. Estos problemas proporcionan un marco de modelado flexible y eficiente para formular y resolver muchos problemas de ingeniería.

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.1 Introducción

- **Paradójicamente** un modelo de **programación entera** pura o entera binaria tiene un número finito de soluciones mientras que un modelo de **programación lineal** tiene (en general) infinitas soluciones al modelo

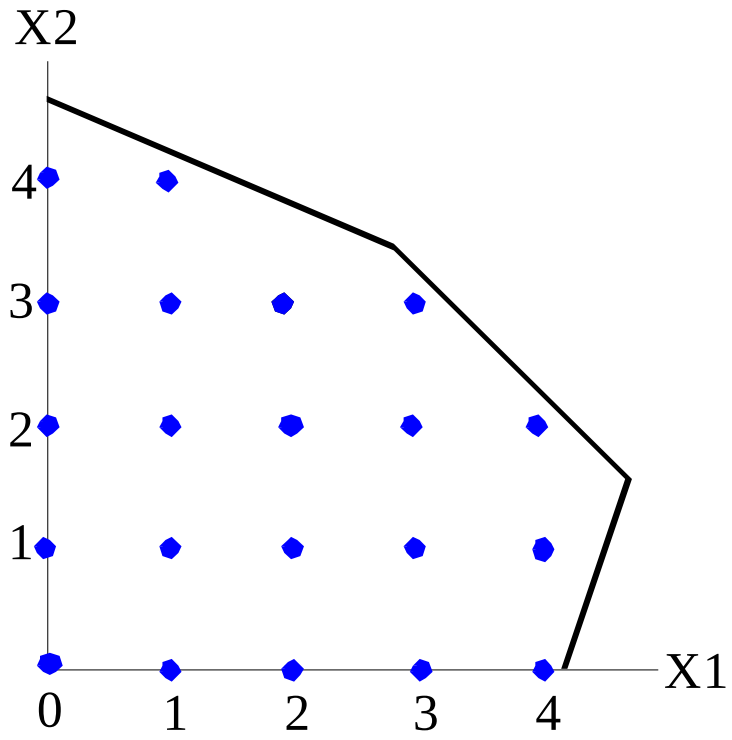
¡¡¡ MIL MILLONES DE SOLUCIONES

ES UN NÚMERO FINITO !!!

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.1 Introducción

- **Un modelo de programación entera NO cumple la propiedad de convexidad:**



© P.Tormos – A.Lova (Universitat Politècnica de València)

5.1 Introducción

Un problema entero sencillo para desconfiar de redondeos

- El **departamento de I+D** de una gran empresa dispone de 2.5 millones de € para adquirir un nuevo equipamiento informático y resulta imposible ampliar esta cantidad con fondos provenientes de otras fuentes. Diversos estudios realizados indican que sólo dos tipos de máquinas son adecuados y que cualquier número o combinación de ellas sería aceptable. Se han realizado unas pruebas para evaluar la capacidad de carga en unidades de "**trabajos promedio**" por hora para los dos tipos de máquinas

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.1 Introducción

Máquina	Coste (millones de €)	Capacidad (por hora)
1	1.4	28 trabajos
2	0.6	11 trabajos

- El **objetivo** del departamento de I+D es **maximizar la capacidad potencial de trabajo**.

Es evidente que ***las máquinas sólo pueden adquirirse en unidades enteras***

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.1 Introducción

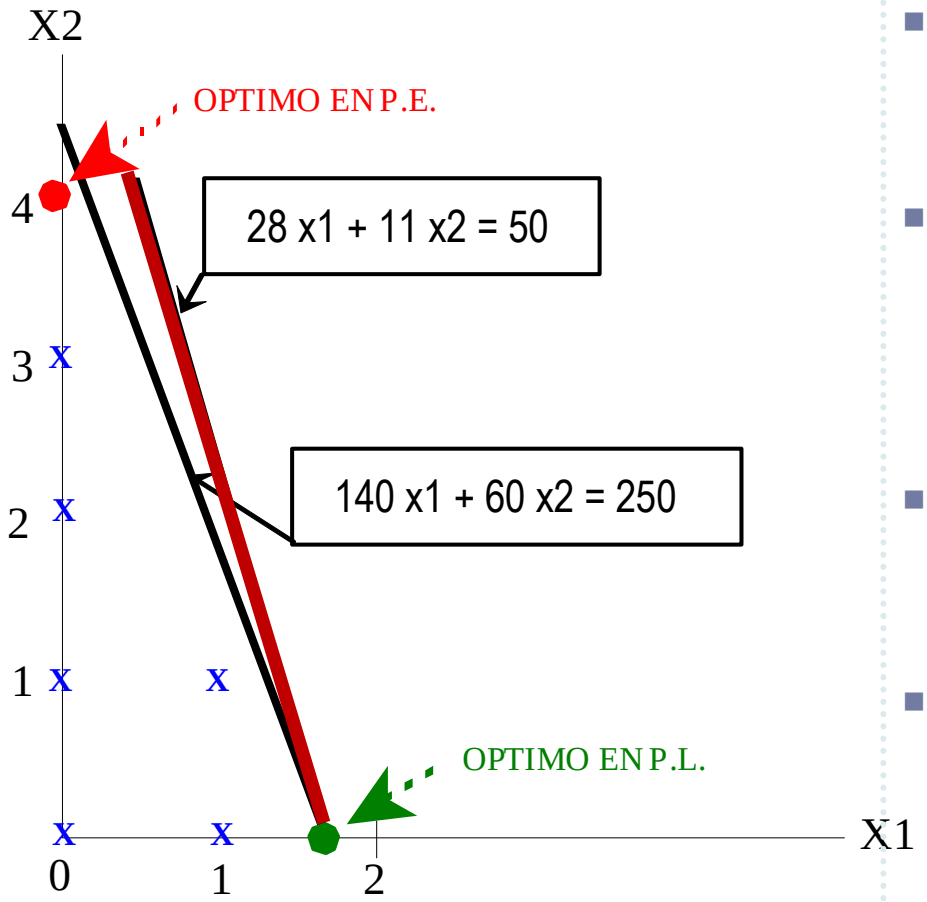
- **VARIABLES:**
  - X_1 : número de máquinas de tipo 1
  - X_2 : número de máquinas de tipo 2
- **FUNCIÓN OBJETIVO:**
  - Maximizar la capacidad de trabajos por hora (en trabajos promedio)

    Max $Z = 28\,X_1 + 11\,X_2$
- **RESTRICCIONES:**
  - Recursos disponibles: $14\,X_1 + 6\,X_2 \leq 25$
  - Condiciones de no negatividad: $X_1, X_2 \geq 0$
  - Condición de enteros: **X_1, X_2 enteros**

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.1 Introducción

- REGIÓN FACTIBLE Y SOLUCIÓN ÓPTIMA:



- **Solución óptima continua**:

  X_1=1.78, X_2=0 y **Z=50**

- **Redondear al entero más cercano:**

  X_1=2, X_2=0 y Z=56

  (**Solución no factible**)

- **Solución entera factible más próxima:**

  X_1=1, X_2=0 y **Z=28**

- **Solución entera óptima:**

  X_1=0, X_2=4 y Z=44

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.1 Introducción

- En la resolución de un problema de programación lineal entera (PPLE) es clave el concepto de **Relajación Lineal**.
- La **Relajación Lineal** de un PPLE es el PL que se obtiene **omitiendo todas las restricciones sobre el carácter entero de las variables.**
- La Relajación Lineal es menos restringida que el PPLE, es decir, *la región factible del PPLE está contenida en la región factible de la correspondiente Relajación Lineal*. Esto implica que para cualquier PPLE de maximización se cumple:

Valor óptimo $\boldsymbol{z}$ *de la Relajación Lineal* $\geq$ *Valor óptimo de* $\boldsymbol{z}$ *de PPLE*

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.1 Técnicas de programación entera: algoritmos de planos de corte

- Ralph **Gomory** fue el creador del primer algoritmo para resolver modelos de programación entera (1958):

Algoritmos de planos de corte

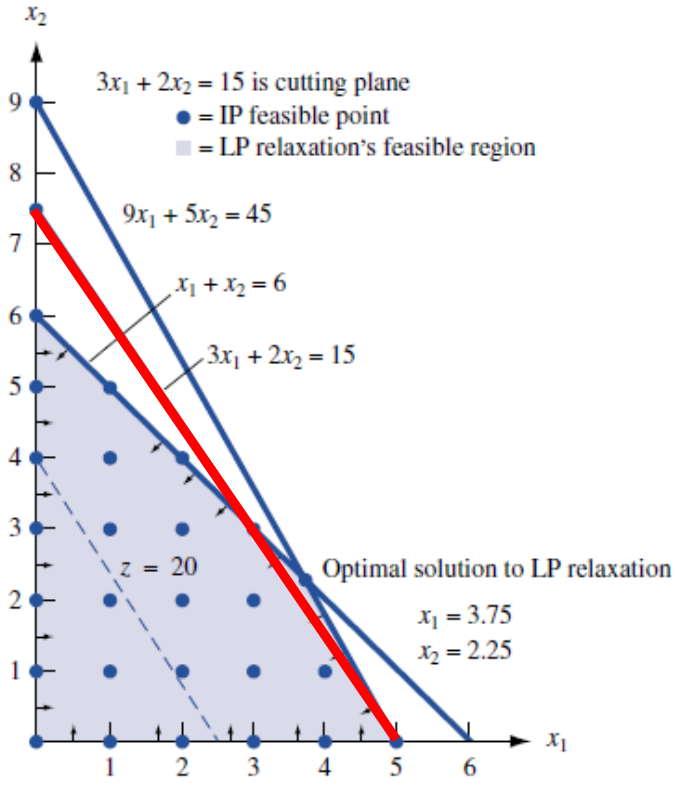
- El **algoritmo de Gomory** consiste en resolver el problema sin considerar las restricciones de carácter entero de las variables. Si la solución no es entera añade restricciones que reducen el conjunto de soluciones del problema lineal continuo asociado sin excluir ninguna solución entera.

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.1 Técnicas de programación entera: algoritmos de planos de corte

- Para cada nueva restricción *(plano de corte)*:

1. Cualquier solución posible la cumple.
2. La solución óptima actual de la relajación lineal del problema **NO** la cumple.



Gomory demostró que este algoritmo encuentra la solución óptima del PPLE después de un número finito de cortes (restricciones)

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

- **Land** y **Doig** (1960): algoritmos de "**bifurcación y acotación**" o técnicas "**branch and bound**"- **B&B**
- **Branch & Bound:** Grupo de procedimientos que realizan la enumeración de las soluciones de forma inteligente de modo que no sea necesario examinar todas las combinaciones de valores de las variables.
- B&B tiene dos cualidades destacables:
  1. Puede aplicarse esencialmente del mismo modo a problemas de programación lineal entera -PPLE (PE Pura o PE Mixta)
  2. Típicamente obtiene una sucesión de soluciones factibles de modo que, si es necesario interrumpir el proceso de búsqueda, la mejor solución hasta el momento puede ser aceptable como una solución aproximada.

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

- Es la técnica más utilizada en la práctica para resolver PPLE.
- **B&B** busca la solución óptima enumerando de ***forma eficiente*** las soluciones posibles del problema.
- OBSERVACIÓN: si se resuelve la relajación lineal de un PPLE pura y se obtiene una solución en la que todas las variables toman valor entero, entonces la solución óptima de la relajación lineal es también solución óptima del PPLE.
- **B&B** comienza resolviendo la relajación lineal del PPLE. Si la solución es entera, esta solución será la óptima del problema original (ver OBSERVACIÓN). En otro caso, se cumplirá (si max):

Valor óptimo $\boldsymbol{z}$ *de la Relajación Lineal* $\geq$ *Valor óptimo* $\boldsymbol{z}$ *de PPLE*

Valor Optimo z de la Relajación Lineal es una cota superior del Valor óptimo de z de PPLE

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

- Las técnicas B&B comienzan resolviendo el problema original como un PL (i.e. relajando las condiciones de que las variables deben tomar valores discretos)
- Si no se obtiene solución entera, una de las variables enteras que todavía tiene valor continuo, denotada por $x_s = a, b$ se selecciona y se crean **dos** descendientes del problema original, uno en el que $x_s \geq a + 1$ y otro en el que $x_s \leq a$. Esta operación se denomina ***Ramificación (o Bifurcación)***.
- El proceso se repite seleccionando uno de los problemas como el actual problema de PE y tratándolo exactamente igual que el original. Esto a su vez genera **dos** nuevos problemas que reemplazan al anterior a menos que por ejemplo el problema en curso no tenga solución posible.
- Repitiendo iterativamente el proceso se alcanza una solución entera (si existe) para uno de los actuales problemas que se convierte en **candidata** para ser la solución óptima del problema original. La mejor de estas soluciones candidatas es una forma de eliminar descendientes del problema original que es inútil explorar porque no conducirían a la solución óptima. La mejor solución entera candidata se llama ***incumbente*** y el proceso de eliminación se denomina ***Poda***.

© P.Tormos – A.Lova (Universitat Politècnica de València)

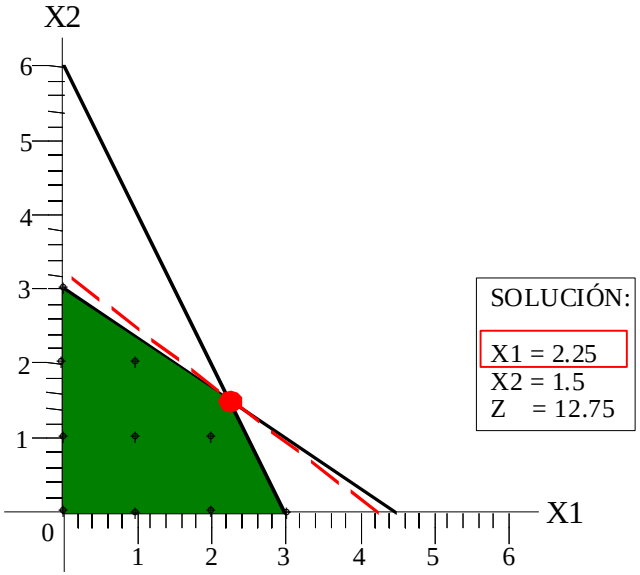
5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

- La representación de todos los subproblemas que se crean se llama árbol.
- Cada subproblema se denomina nodo del árbol
- Cada línea que conecta dos nodos del árbol es un arco o rama.
- El subproblema correspondiente a cada nodo del árbol está compuesto por la relajación lineal del PPLE + las restricciones asociadas a los arcos que llevan del subproblema inicial hasta este nodo.

© P.Tormos – A.Lova (Universitat Politècnica de València)

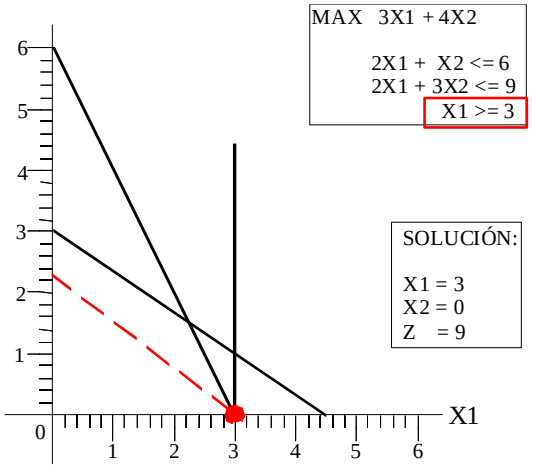
5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

En la figura se muestra la región factible de un problema de Programación Lineal en el que x1 y x2 han de ser enteras. La solución óptima se ha obtenido sin tener en cuenta dicha condición:

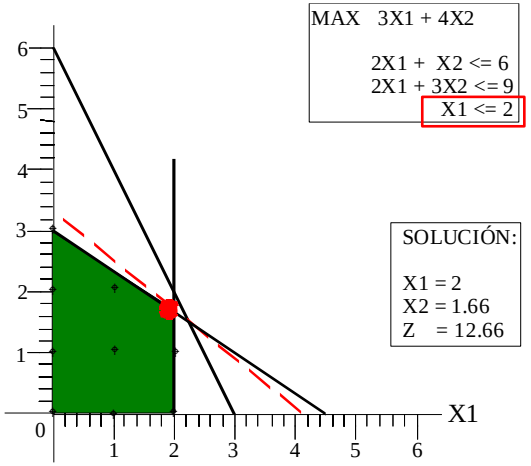


Dado que la Solución Optima requiere que tanto x1 como x2 sean enteras y todavía no lo son, el problema se divide en dos nuevos subproblemas en los que x1 toma valor entero:

1



2



El proceso se repite hasta que se alcanza una *hoja* del árbol de soluciones.

© P.Tormos – A.Lova (Universitat Politècnica de València)

Resolución Gráfica

- Para exponer el algoritmo de Bifurcación y Acotación, utilizaremos el siguiente modelo lineal:

(P_0):

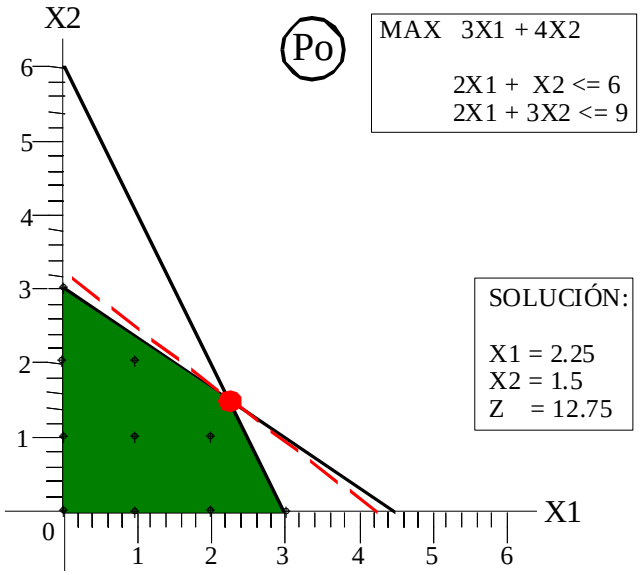
Max 3 x1 + 4 x2

s.a:

$2\,x1 + x2 \leq 6$

$2\,x1 + 3\,x2 \leq 9$

$x1, x2 \geq 0$ **y** x1, x2 **enteras**



© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

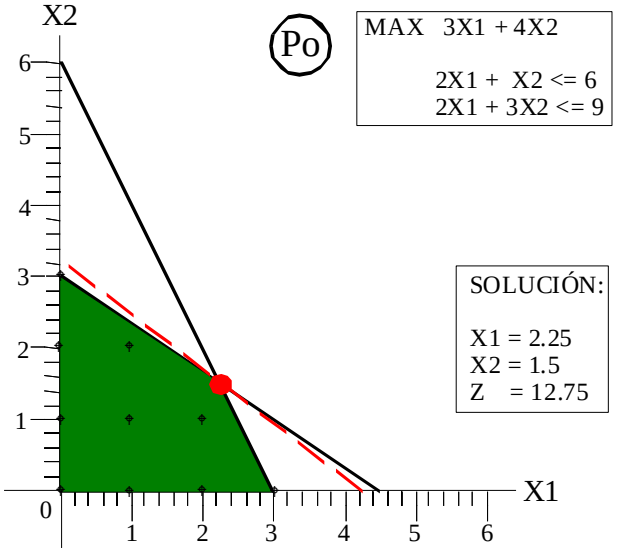
Cota **inferior** de la Función Objetivo en la Solución Óptima ENTERA → **$Z^* = -\infty$**

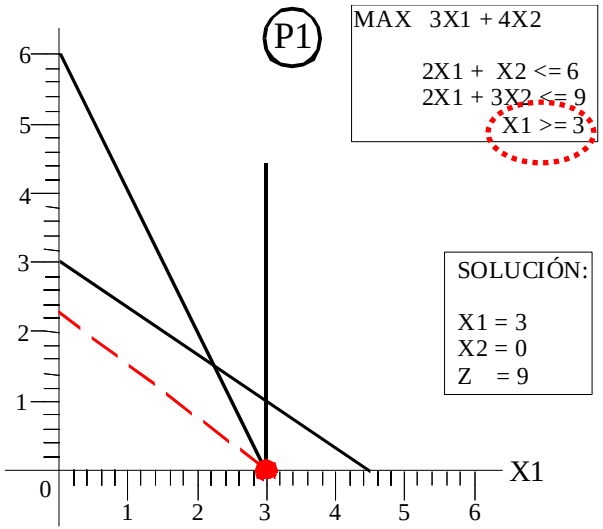
P0

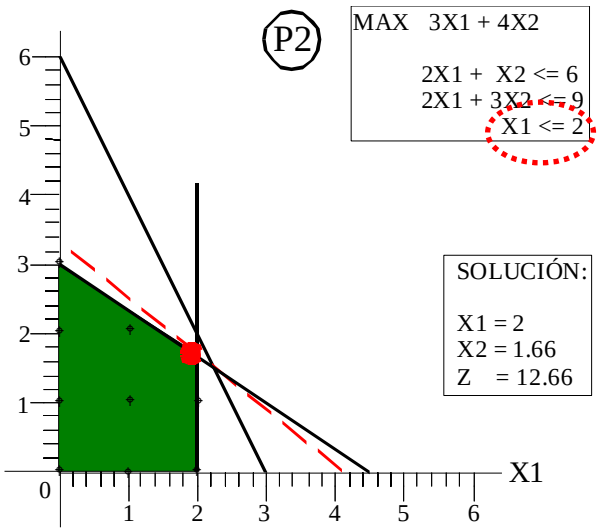
X1=2.25
X2=1.5
Z_0=12.75
$Z^* = -\infty$

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

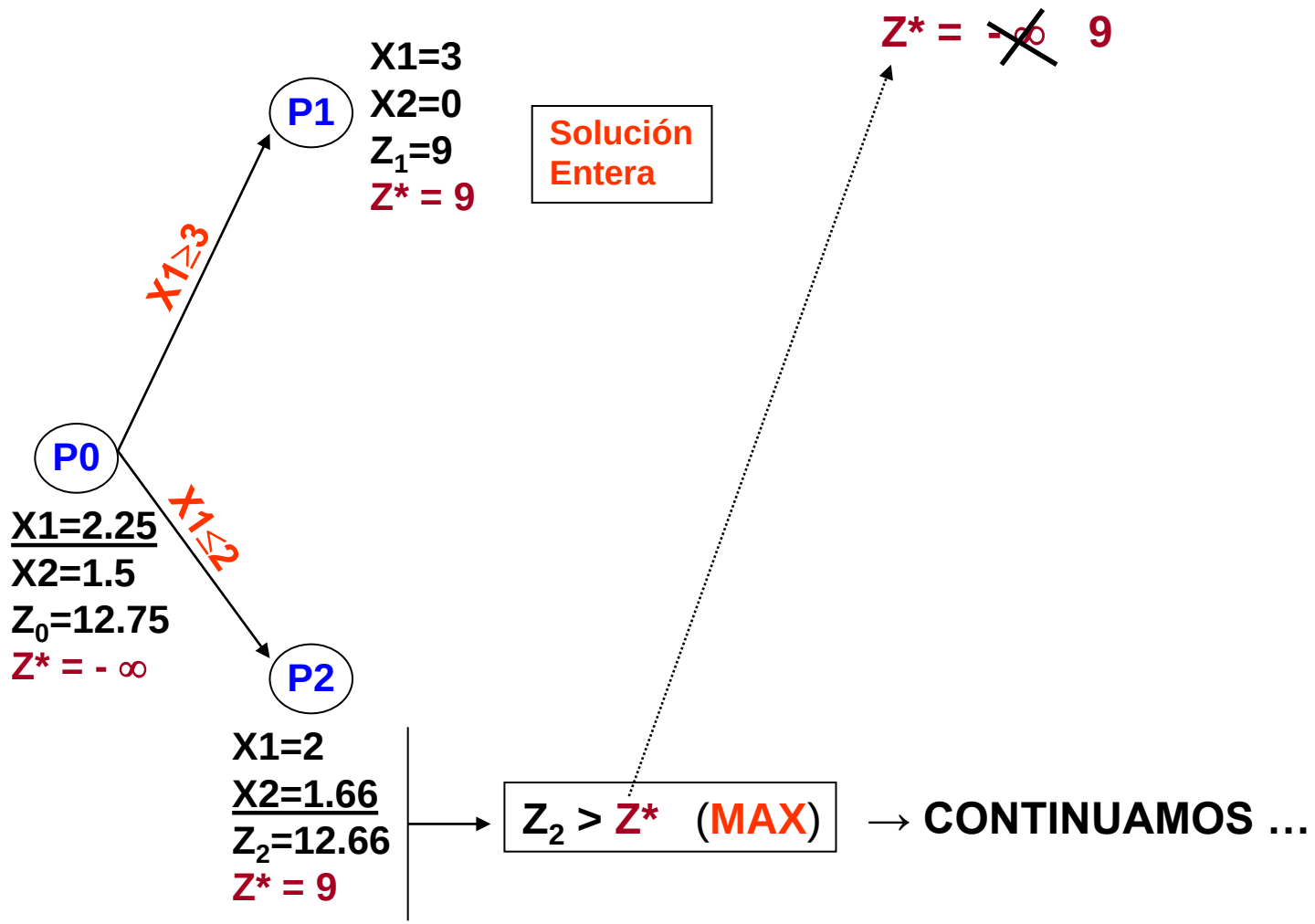






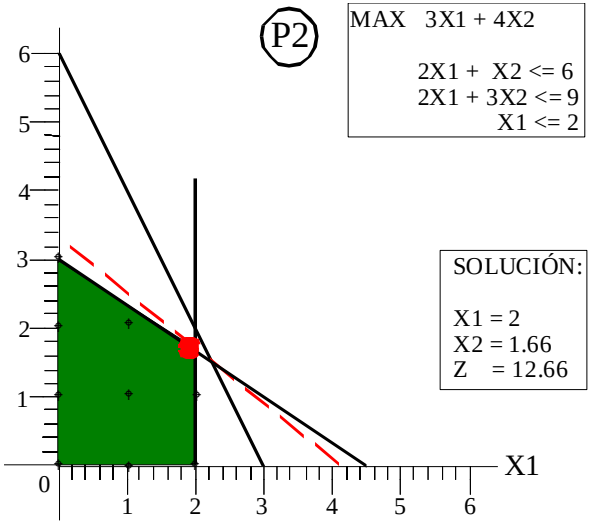
© P.Tormos – A.Lova (Universitat Politècnica de València)

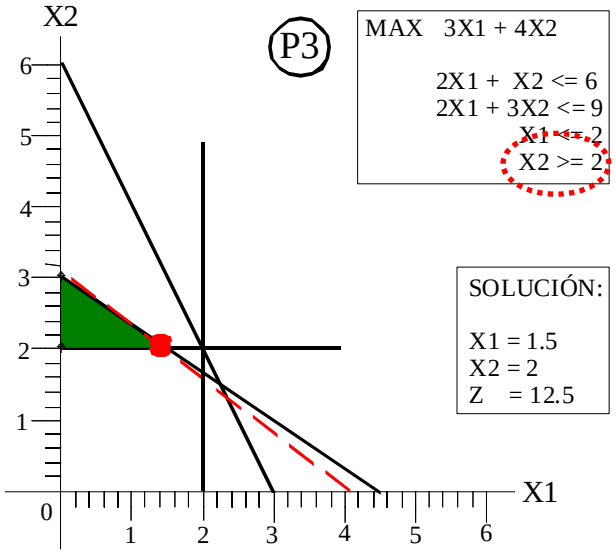
5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

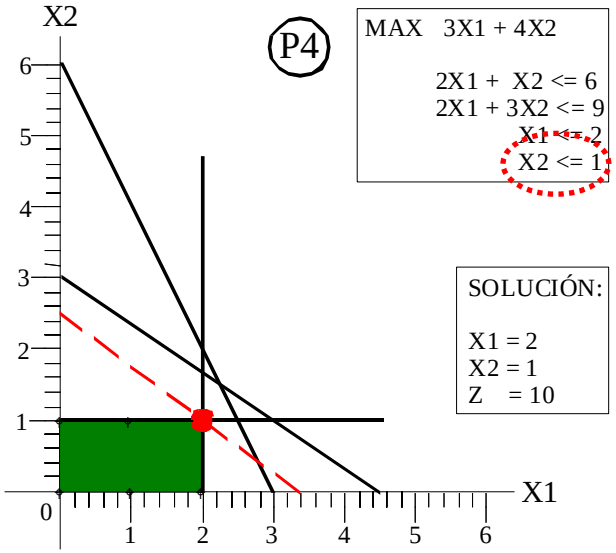


© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

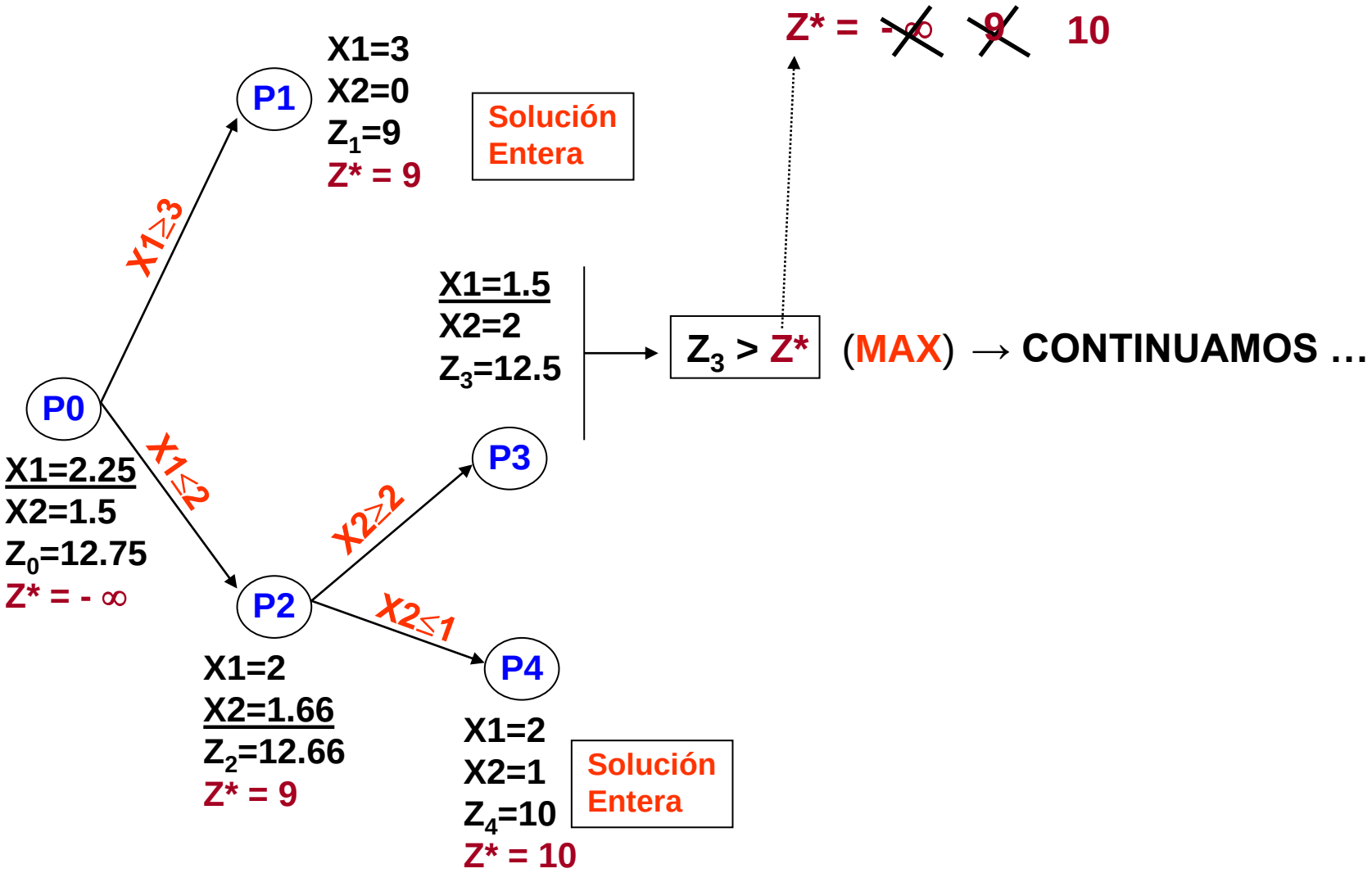






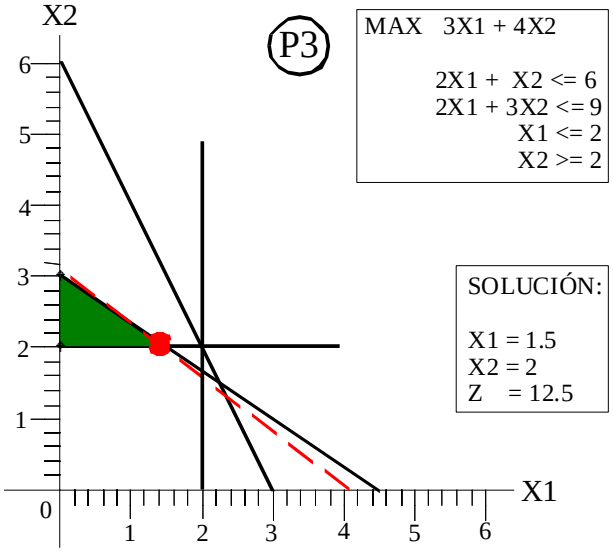
© P.Tormos – A.Lova (Universitat Politècnica de València)

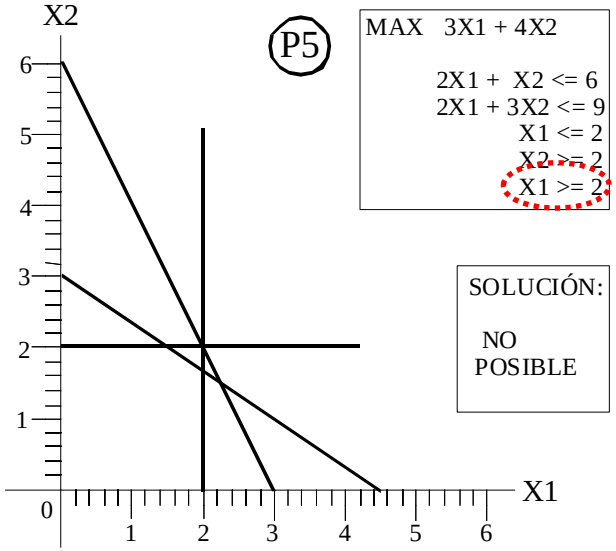
5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

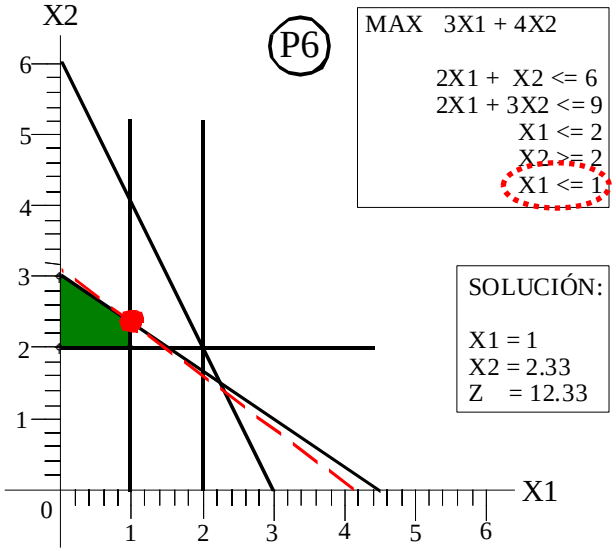


© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

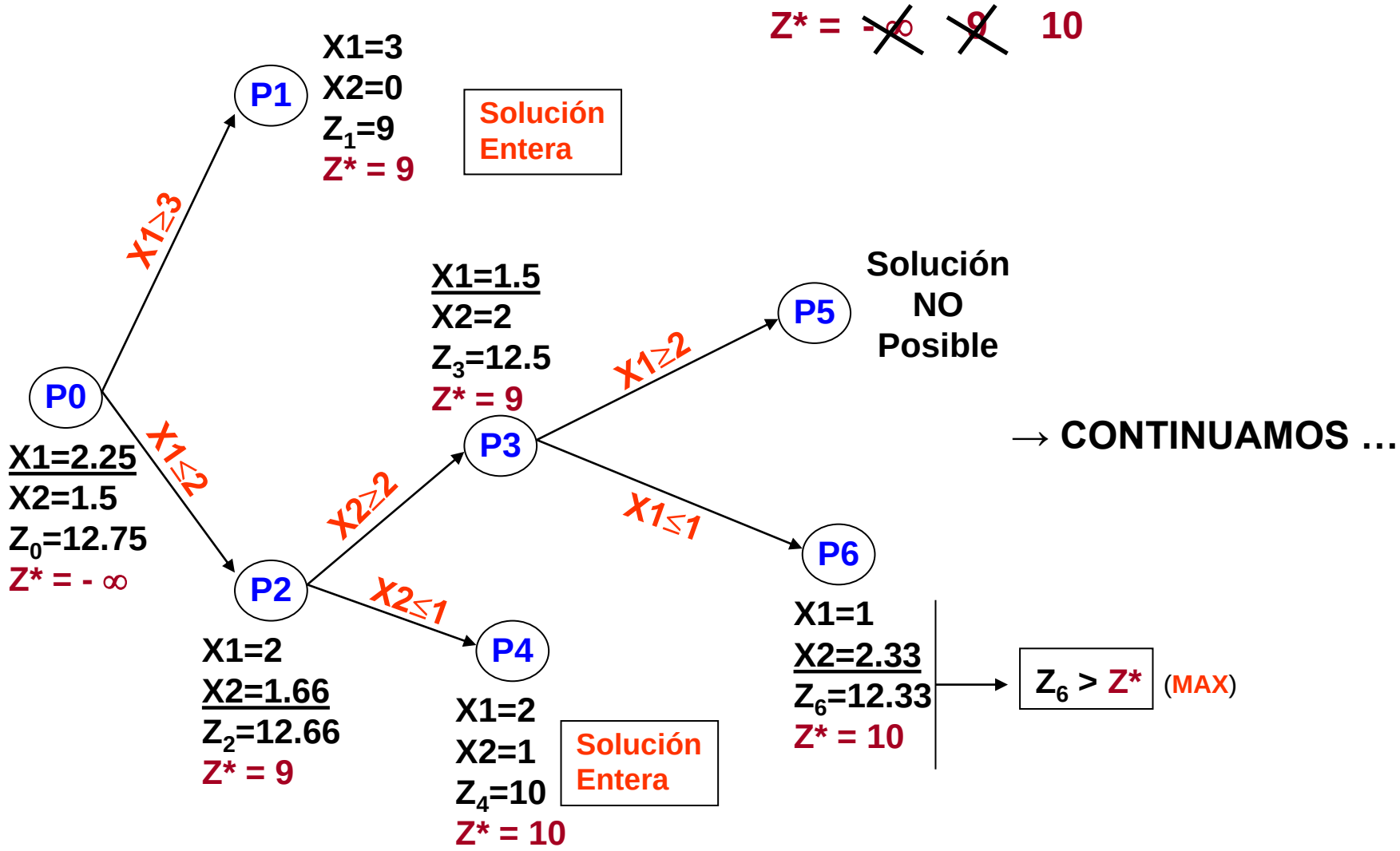






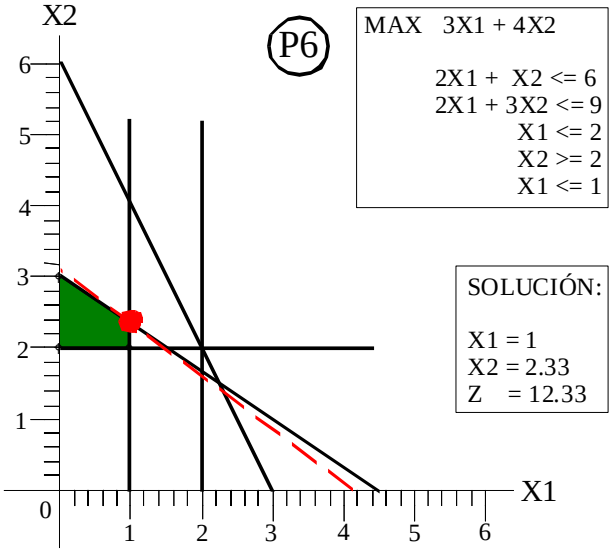
© P.Tormos – A.Lova (Universitat Politècnica de València)

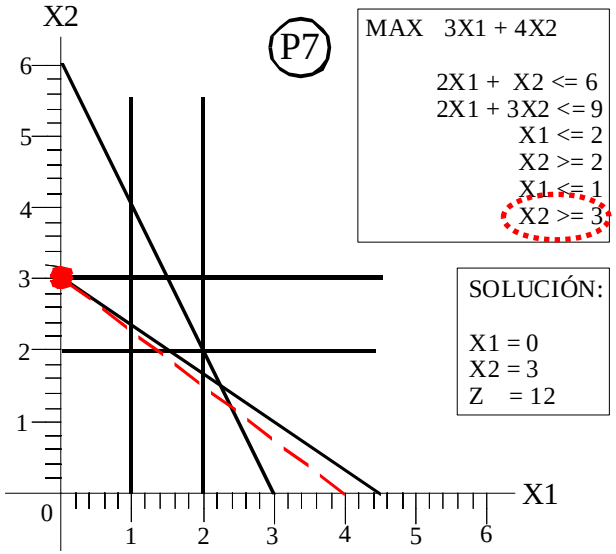
5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

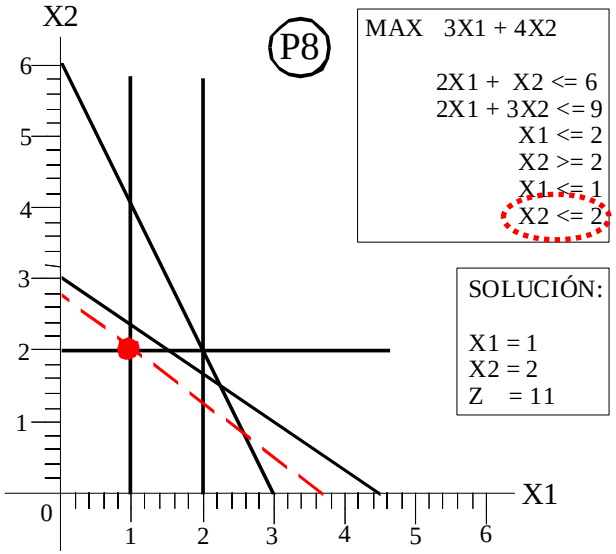


© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

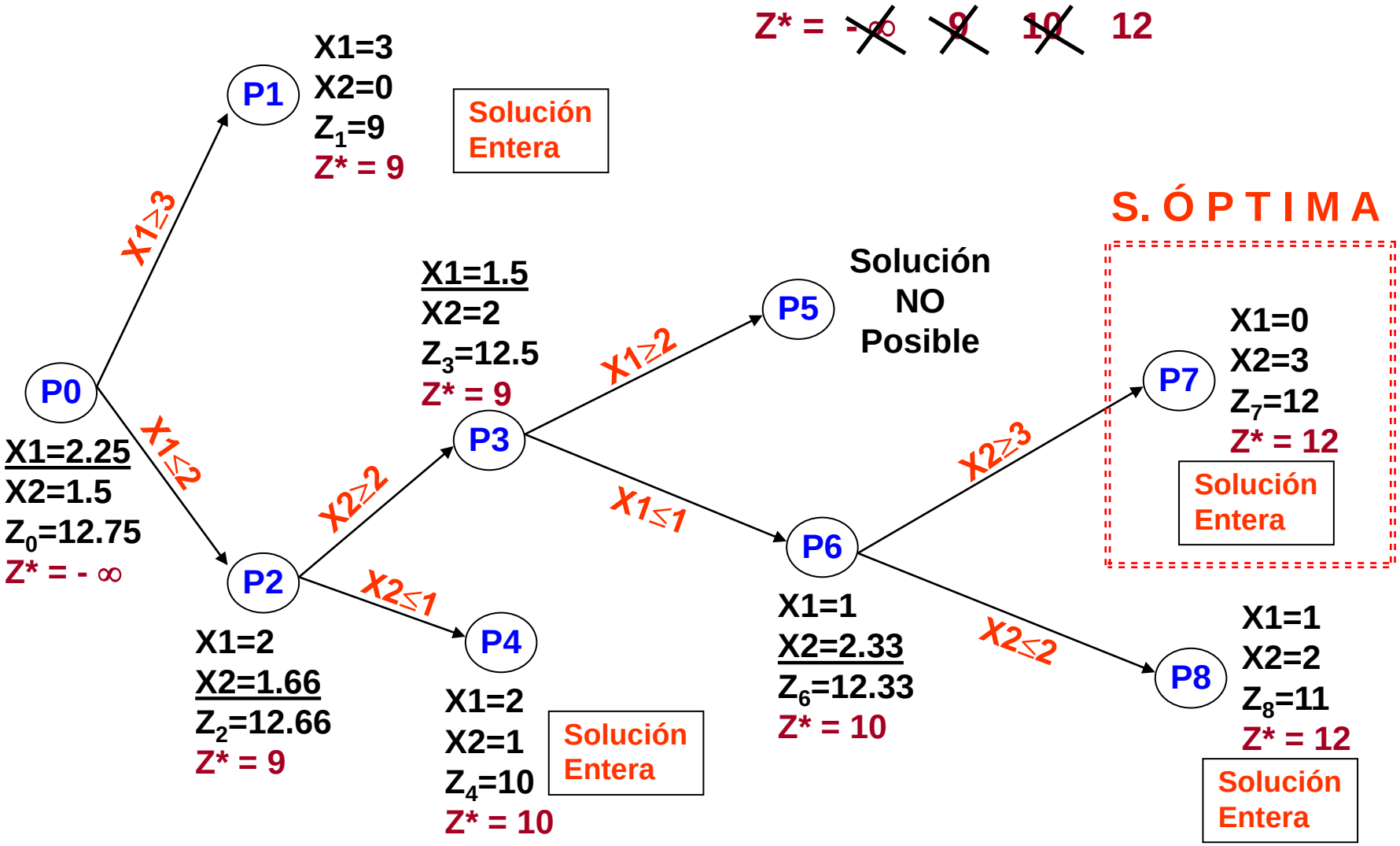






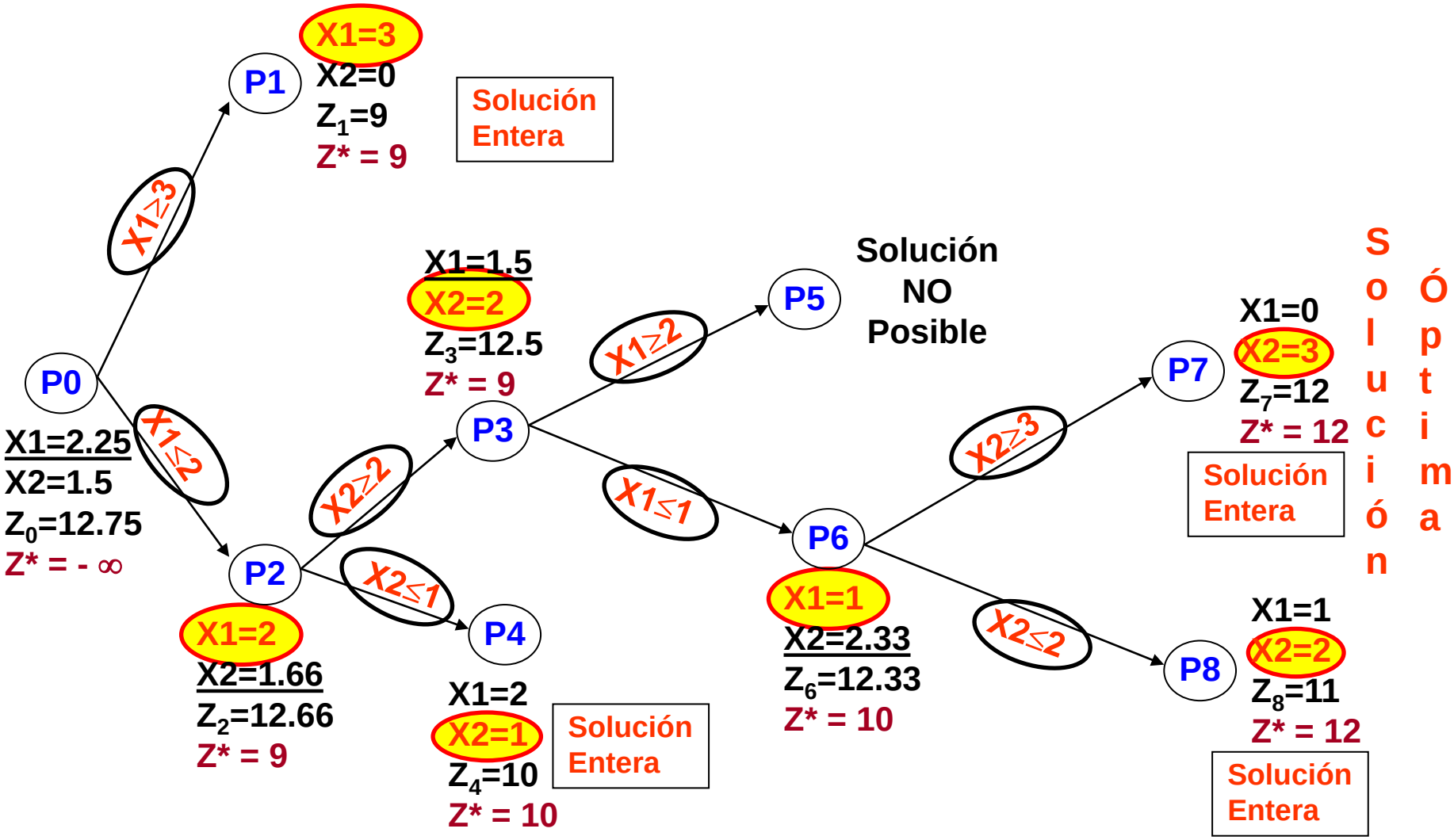
© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación



© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación



© P.Tormos – A.Lova (Universitat Politècnica de València)

Estrategias de poda:

- La **poda** de la rama correspondiente tiene lugar por una de las tres razones siguientes:

1. La solución obtenida satisface las condiciones de integralidad (las variables que han de ser enteras, tienen valor entero).
2. El problema considerado NO tiene solución.
3. La solución del problema es continua y el valor de la función objetivo es peor que la mejor cota entera disponible.

(En los 3 casos anteriores, el nodo correspondiente será una **hoja** del árbol de soluciones y por tanto no se bifurcará de nuevo)

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

¿Qué nodo elegir para hacer la siguiente bifurcación?

1. **Técnica de la mejor cota (jumptracking)**

   Consiste en elegir el nodo con mejor valor óptimo continuo, pretendiendo con ello encontrar **buenas soluciones factibles**. Cuando se bifurca un nodo esta aproximación resuelve todos los subproblemas creados.

   Después el proceso continúa bifurcando de nuevo el nodo con mejor valor de la función objetivo.

   En problemas grandes, este criterio suele conducir a tener que almacenar muchos datos y presenta **problemas de memoria** de ordenador.

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

¿Qué nodo elegir para hacer la siguiente bifurcación?

2. **Técnica de la cota más reciente (backtracking)**

   Este criterio consiste en elegir el nodo de creación más reciente. De esta forma se avanza en profundidad en el árbol y, aunque haya que examinar más nodos que con el criterio anterior (no se tiene en cuenta la función objetivo), es más fácil poder eliminarlos durante el proceso.

   Requiere **poca memoria** de ordenador

© P.Tormos – A.Lova (Universitat Politècnica de València)

5.2.2 Técnicas de programación entera: algoritmo de bifurcación y acotación

PASOS DEL ALGORITMO BIFURCACIÓN Y ACOTACIÓN (B&B)

1. Resolver el problema / nodo raíz resultante de relajar la condición de variables enteras.
2. Comprobar si se cumple alguno de los criterios para saturar ese nodo y si hay que actualizar Z*
3. Si el nodo queda abierto, elegir una variable por la cual bifurcar
4. Elegir el siguiente nodo a resolver de entre los que siguen abiertos y repetir con él los pasos 1-3.
5. El algoritmo termina cuando todos los nodos han sido visitados. La SOLUCIÓN OPTIMA ENTERA es la mejor de todas las soluciones enteras encontradas al recorrer el árbol.

© P.Tormos – A.Lova (Universitat Politècnica de València)

Ejercicio 1:

Considera el siguiente problema lineal entero:

$$\left.\begin{array}{ll} \text{Max} & z = 3x_1 + 4x_2 + 5x_3 \\ \text{s.a:} & 2x_1 + x_2 + x_3 \leq 6 \\ & 2x_1 + 3x_2 + 4x_3 \leq 9 \\ & x_1 + 3x_2 + 2x_3 \leq 7 \\ & x_1, x_2, x_3 \geq 0 \\ & x_1, x_2, x_3 \text{ enteras} \end{array}\right\}.$$

Se ha aplicado el algoritmo de bifurcación y acotación (branch and bound) para encontrar la solución óptima de este problema. Las iteraciones resultantes se muestran en la página siguiente. A partir de la información proporcionada por el algoritmo, responde a las siguientes cuestiones de forma razonada.

a) Representa en forma de árbol los pasos seguidos por el algoritmo. Indica claramente qué solución representa cada nodo del árbol y a qué iteración corresponde, y qué cota es la que se aplica en cada arista o rama.

b) De acuerdo con las normas que rigen el algoritmo, ¿debería haberse bifurcado el nodo (P9)? Justifica tu respuesta.

c) Explica qué problema lineal se ha resuelto en la iteración (P6).

d) Si se hubiera seguido el criterio de la «mejor cota», ¿cuál habría sido la secuencia de visita de los diferentes problemas?

© P.Tormos – A.Lova (Universitat Politècnica de València)

(P_1)	z = 12,75		
Variable	Valor	Cota inf.	Cota sup.
x_1	2,25	0	M
x_2	1,5	0	M
x_3	0	0	M

(P_2)	z = 9		
Variable	Valor	Cota inf.	Cota sup.
x_1	3	3	M
x_2	0	0	M
x_3	0	0	M

(P_3)	z = 12,667		
Variable	Valor	Cota inf.	Cota sup.
x_1	2	0	2
x_2	1,667	0	M
x_3	0	0	M

(P_4)	z = 11		
Variable	Valor	Cota inf.	Cota sup.
x_1	1	0	2
x_2	2	2	M
x_3	0	0	M

(P_5)	z = 12,5		
Variable	Valor	Cota inf.	Cota sup.
x_1	2	0	2
x_2	1	0	1
x_3	0,5	0	M

(P_6)	z = 12,33		
Variable	Valor	Cota inf.	Cota sup.
x_1	2	0	2
x_2	0,333	0	1
x_3	1	1	M

(P_7)	z = 12		
Variable	Valor	Cota inf.	Cota sup.
x_1	1	0	2
x_2	1	1	1
x_3	1	1	M

(P_8)	z = 12,25		
Variable	Valor	Cota inf.	Cota sup.
x_1	2	0	2
x_2	0	0	0
x_3	1,25	1	M

(P_9)	z = 11,5		
Variable	Valor	Cota inf.	Cota sup.
x_1	0,5	0	2
x_2	0	0	0
x_3	2	2	M

(P_{10})	z = 11		
Variable	Valor	Cota inf.	Cota sup.
x_1	2	0	2
x_2	0	0	0
x_3	1	1	1

(P_{11})	z = 10		
Variable	Valor	Cota inf.	Cota sup.
x_1	2	0	2
x_2	1	0	1
x_3	0	0	0

© P.Tormos – A.Lova (Universitat Politècnica de València)

Ejercicio 2:

Se ha resuelto mediante el algoritmo Branch-and-Bound el siguiente problema de Programación Lineal Entera Mixta:

$$\begin{array}{ll} \text{Max} & 12\delta_1 + 35\delta_2 + 20\delta_3 + 2x \\ & \left.\begin{array}{r} 4\delta_1 + 4\delta_2 + \delta_3 \leq 6 \\ 10\delta_2 + 9\delta_3 + 1{,}5x \leq 12 \\ 1{,}5\delta_2 + 1{,}5\delta_3 \leq 7 \\ x \geq 0 \\ \delta_1\,,\,\delta_2\,,\,\delta_3 \text{ binarias} \end{array}\right\} \\ \text{s.a:} & \end{array}$$

a) A partir de la información facilitada, reproduce el árbol generado por el algoritmo Branch-and-Bound. Especifica qué cota o restricción representa cada rama, y qué solución se obtiene en cada nodo.

b) De acuerdo con el funcionamiento del algoritmo Branch-and-Bound, explica si hay en este árbol ramas que no deberían haberse generado y si, por otro lado, falta bifurcar algún nodo. JUSTIFICA tu respuesta en cualquier caso.

c) Si la información proporcionada por el algoritmo lo permite, indica cuál es la solución óptima del problema y si ésta es única; en caso contrario, señala la mejor solución entera encontrada.

d) Explica qué árbol se habría obtenido (y en qué orden se habrían generado los nodos) si el criterio para recorrer el árbol hubiera sido el de la "mejor cota".

e) ¿Qué problema se ha resuelto en el nodo (P6)? Especifica claramente su función objetivo y sus restricciones.

© P.Tormos – A.Lova (Universitat Politècnica de València)

(P_1)	Valor f.obj. = 44,7778		
Variable	Valor	Cota inf.	Cota sup.
δ_1	0,4444	0	1
δ_2	1	0	1
δ_3	0,2222	0	1
x	0	0	M

(P_2)	Valor f.obj. = 41,4167		
Variable	Valor	Cota inf.	Cota sup.
δ_1	1	1	1
δ_2	0,25	0	1
δ_3	1	0	1
x	0,3333	0	M

(P_3)	Problema imposible		
Variable	Valor	Cota inf.	Cota sup.
δ_1		1	1
δ_2		1	1
δ_3		0	1
x		0	M

(P_4)	Valor f.obj. = 36		
Variable	Valor	Cota inf.	Cota sup.
δ_1	1	1	1
δ_2	0	0	0
δ_3	1	0	1
x	2	0	M

(P_5)	Valor f.obj. = 39,4444		
Variable	Valor	Cota inf.	Cota sup.
δ_1	0	0	0
δ_2	1	0	1
δ_3	0,2222	0	1
x	0	0	M

(P_6)	Valor f.obj. = 30,5		
Variable	Valor	Cota inf.	Cota sup.
δ_1	0	0	0
δ_2	0,3	0	1
δ_3	1	1	1
x	0	0	M

(P_7)	Valor f.obj. = 37,6667		
Variable	Valor	Cota inf.	Cota sup.
δ_1	0	0	0
δ_2	1	0	1
δ_3	0	0	0
x	1,3333	0	M

© P.Tormos – A.Lova (Universitat Politècnica de València)

Fuentes:

- Tormos, P. y Lova, A.: «Investigación Operativa para Ingenieros». SPUPV-2003.591
- Tormos, P. y Lova, A.: «Diseño de Sistemas de Optimización basados en Hoja de Cálculo». Apuntes docentes. DEIOAC-UPV. 2014

© P.Tormos – A.Lova (Universitat Politècnica de València)